
Globe Valve GAD Automation

Technical Documentation

Design Software - Flask + SolidWorks COM Automation

1. Overview

The Globe GAD Automation module generates a General Arrangement Drawing (GAD) for Series 10/11 Globe control valves directly from a valve Bill of Materials (BOM) row, driving SolidWorks over COM automation. It is a Python/Flask re-implementation of logic that previously lived in a standalone WinForms desktop application, now exposed as a web page any authorized user can run from a browser.

Two output files are produced for every generation: a 3-page PDF drawing and the native SolidWorks drawing file (.SLDDRW), delivered together as a single .zip download.

Generate on Server

The single supported generation path runs entirely on whichever machine hosts the Flask app, provided that machine has SolidWorks installed and licensed. Clicking **Generate on Server** kicks off a background job on that machine; progress is streamed to the browser step by step in real time until both output files are ready to download.

2. System Architecture

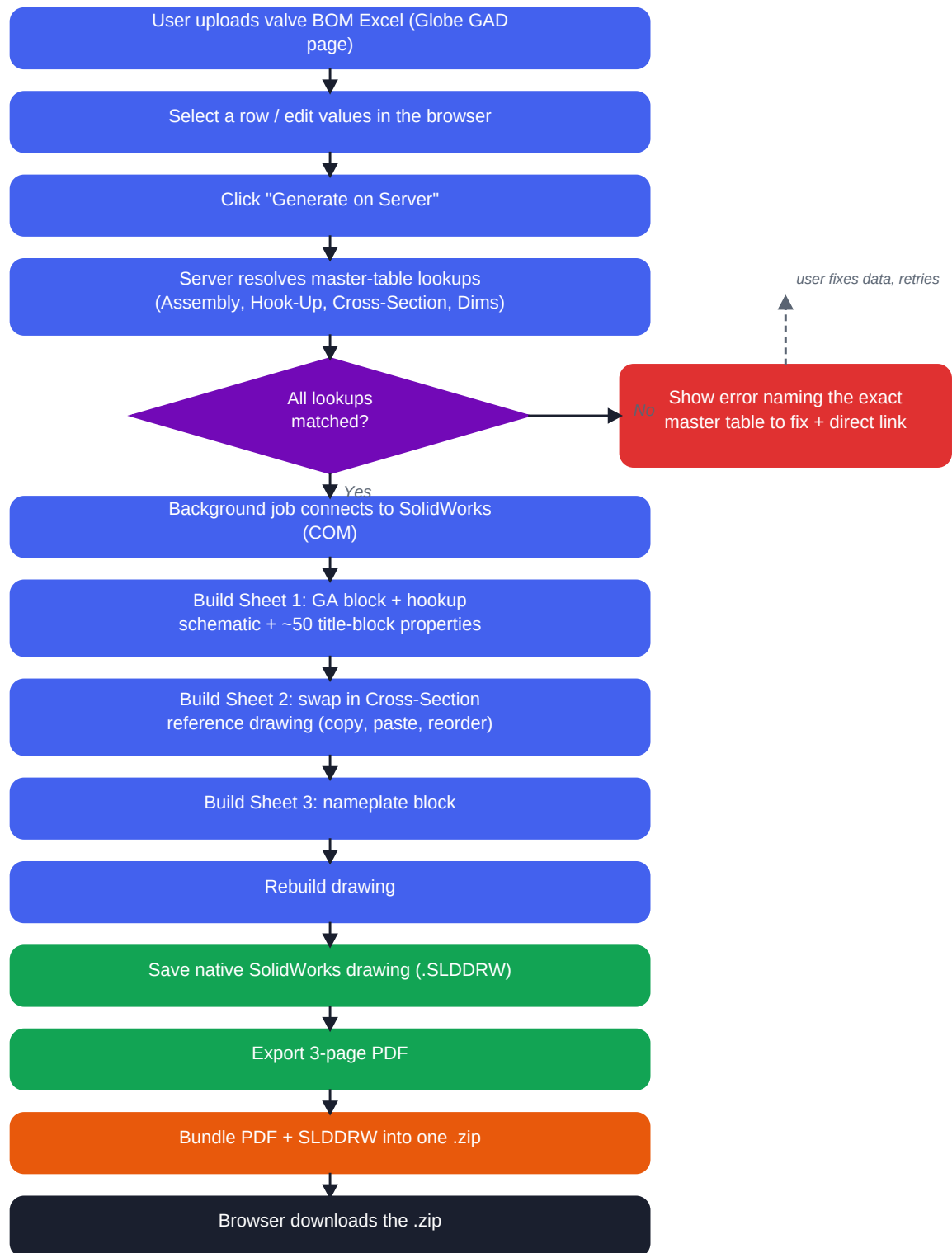
The application is a standard Flask web app backed by PostgreSQL, with a dedicated automation layer that talks to SolidWorks through the pywin32 COM bridge. The architecture keeps database access and SolidWorks automation cleanly separated:

Layer	Responsibility	Key file(s)
Web / UI	Upload BOM, row editor, progress UI, downloads	templates/globe.html
Routes	HTTP endpoints, background job orchestration	app/blueprints/dashboard/routes.py
Lookup resolution	Master-table queries -> one flat resolved dict (no SolidWorks dependency)	app/services/gad_globe_lookup.py
SolidWorks automation	COM automation building the drawing (no database dependency)	app/services/solidworks_automation.py
Progress tracking	In-memory job/step state for the browser to poll	app/services/gad_progress.py
Master data admin	Upload/view/manage the 5 lookup tables	app/services/gad_masters_import.py
Database	PostgreSQL - BOM master lookup tables	app/models.py

This split - resolve first, then automate - keeps the SolidWorks automation layer free of any database dependency: it operates purely on the already-resolved dict handed to it, which makes that layer straightforward to test and reason about in isolation from the web/database side of the app.

3. End-to-End Process Flow

The diagram below traces one "Generate on Server" click from the browser through to the final downloaded file, including the master-data validation loop.



4. Master Data Tables

Every generation resolves the incoming BOM row against 5 lookup tables, each manageable through the "Globe GAD Masters" admin pages (upload an Excel sheet, view/filter existing rows). A row that doesn't match any master-table entry fails generation immediately with an error naming the exact table to fix.

Master table	Model / DB table	Feeds	Key columns
Overall Assembly	GAGlobeTable ga_globe_table	Sheet 1 - GA block	valve_series, body_style, valve_size, rating, end_connection, bonnet_type, flow_direction, actuator_series/type/size, travel_stop, hw -> drawing_no
Hook-Up Selection	GAGlobeHookUp ga_globe_hookup	Sheet 1 - hookup schematic	actuator, spring, air_fail_action, positioner, hand_auto, limit_switch, position_trans, volume_booster, spool_valve, lock_valve, solenoid_valve -> schematic_no
Cross-Section (formerly "Sheet4 Selection")	GASheet4Globe ga_sheet4_globe	Sheet 2 - whole reference drawing	body_style, end_connection, size, rating, bonnet_type, trim_type, balancing, flow_direction, bal_seal_type, seat_type, packing_type -> drawing_no (CS0xx.SLDDRW)
Valve Dimensions	GADimValveGlobe ga_dim_valve_globe	Title-block A/B/C/AR + weight	body_style, end_connection, bonnet_type, end_finish, series, size, rating, stem_dia
Actuator Dimensions	GADimActGlobe ga_dim_act_globe	Title-block D/E/F + weight	actuator_type, actuator_size, hand_wheel, travel_stop

Legacy table: GACrossSecGlobe (ga_crosssec_globe) previously fed the old Sheet 2 cross-section block and is no longer queried by generation - Sheet 2 is now built entirely from the Cross-Section table above. It remains in the database unused rather than deleted, in case of a future revert.

Size/Rating matching: the Overall Assembly and Cross-Section tables allow a single cell to hold a comma-separated list of valid values (e.g. size "1,4" or rating "ASME 150,ASME2500") - a BOM row matches if its value appears anywhere in that list, independent of spacing.

5. Drawing Generation Detail

SolidWorks automation opens one shared .DRWDOT template per generation and builds three sheets before rebuilding and exporting. All COM automation runs inside a single global lock, since SolidWorks can only drive one document build at a time.

Sheet 1 - General Arrangement

- Insert the GA block (.SLDBLK, looked up via Overall Assembly) at a fixed position.
- Insert the hookup schematic block (.SLDBLK, looked up via Hook-Up Selection).
- Push ~50 custom properties onto the title block (customer, project, tag no, dimensions A/B/C/AR/D/E/F, weight, materials, and more).

Sheet 2 - Cross-Section reference drawing

Sheet 2 no longer inserts a .SLDBLK block. Instead, the whole reference drawing (a separate .SLDDRW file, e.g. CS001.SLDDRW) is copied in wholesale, replacing the template's own Sheet 2 tab outright:

- Delete the template's existing Sheet 2 tab (selected as a whole SHEET entity, not its contents).
- Open the reference file as its own SolidWorks document.
- Select its sheet as a whole SHEET entity and copy it - the source document must stay open until after the paste completes, or the paste silently fails.
- Switch back to the GAD drawing, paste the sheet in, rename it to "Sheet2", and reorder the sheet sequence so it lands as the actual second page.
- Close the reference document.

Sheet 3 - Nameplate

Inserts a fixed nameplate block (.SLDBLK) - the same file on every generation, not looked up per row.

Finishing

- Rebuild the full drawing (ForceRebuild3).
- Save the native SolidWorks drawing (.SLDDRW).
- Export the 3-page PDF from the same document.
- Bundle both files into one .zip and mark the job downloadable.

6. Progress Tracking & Error Reporting

"Generate on Server" runs the SolidWorks automation in a background thread rather than blocking the HTTP request, so the browser can poll for real step-by-step progress instead of showing one long spinner.

Step	Milestone shown to the user
1	Connecting to SolidWorks
2	Opening GAD template
3-4	Sheet 1: inserting GA block / hookup schematic
5	Sheet 1 complete - title block properties set
6	Sheet 2 complete - reference drawing inserted
7	Sheet 3 complete - nameplate inserted
8	Rebuilding drawing
9	Exporting PDF

If any master-table lookup fails, the error names the exact GAD Masters table responsible (e.g. "No matching row found in the Cross-Section master table...") and the browser shows a direct link to that table's admin page, rather than a generic failure message.

Once generation succeeds, SolidWorks is intentionally left open (visible) so the user can inspect the drawing. It only closes - discarding the in-memory copy, no save prompt - once the user dismisses the success message in the browser, or after a 20-minute safety timeout if the tab is abandoned.

7. Technical Implementation Notes

SolidWorks' COM API has several undocumented quirks that required empirical verification against a live SolidWorks session. These are recorded here for future maintainers.

COM connection (early binding)

pywin32's normal `gencache.EnsureDispatch()` fails against `SldWorks.Application` with "This COM object can not automate the makepy process" - SolidWorks' `GetTypeInfo()` call doesn't behave the way `EnsureDispatch` expects. The fix reads both required type libraries (`sldworks.tlb` and `swconst.tlb`) directly from the registry by their stable GUIDs, builds the wrapper modules explicitly, and creates the instance via a plain `Dispatch()` call instead - avoiding the failing lookup entirely.

Dynamic vs. early-bound return values

Several SolidWorks methods (`NewDocument()`, `GetMathUtility()`) declare a plain `IDispatch` return type, so their results come back only partially usable via plain dynamic dispatch - calling members like `ActivateSheet` or `InsertSketch` on them fails with "Member not found". Each such result is explicitly re-wrapped as its known interface class (`ModelDoc2`, `DrawingDoc`, `MathUtility`) before use.

Copying a sheet from another document

- `PasteSheet` silently returns `False` if the source document was already closed - even immediately after `EditCopy()` - because the "sheet on the clipboard" depends on the source document staying alive, not a plain OS clipboard payload.
- `PasteSheet` always inserts the new sheet immediately after Sheet 1, regardless of which sheet was selected as the anchor - final ordering is fixed explicitly afterward with `ReorderSheets`.
- `ReorderSheets` requires its argument as a `SAFEARRAY` of `BSTR` (`VT_ARRAY | VT_BSTR`); a plain Python list marshals as the wrong `VARIANT` subtype and the call just silently returns `False` instead of reordering.
- Deleting a sheet (`Extension.DeleteSelection2`) disconnects pywin32's cached `ModelDoc2/DrawingDoc` references from the live document ("the object invoked has disconnected from its clients" on the next call) - fresh references must be re-wrapped from `sw_app.ActiveDoc` afterward.

Threading

COM requires the calling thread to have an initialized apartment (`pythoncom.CoInitialize()`) before any `Dispatch()` call - true automatically for a plain script's main thread, but not for a Flask request/background thread. A single process-wide lock also serializes the actual drawing-build steps, since SolidWorks can only drive one document build at a time; it's released before the post-success wait for the user's acknowledgment, so one open drawing doesn't block others.

8. Configuration Reference

All file/folder locations are environment variables (.env), needed only on the machine that actually runs SolidWorks automation.

Variable	Purpose
GAD_TEMPLATE_PATH	The shared .DRWDOT template opened for every generation.
GAD_BLOCK_DIR	Folder of GA blocks (.SLDBLK) for Sheet 1, keyed by drawing_no.
GAD_HOOKUP_BLOCK_DIR	Folder of hookup schematic blocks (.SLDBLK) for Sheet 1.
GAD_NAMEPLATE_BLOCK_PATH	The single fixed nameplate block (.SLDBLK) for Sheet 3.
GAD_SHEET4_DIR	Folder of whole reference drawings (.SLDDRW) for Sheet 2, keyed by the Cross-Section table's drawing_no.
GAD_OUTPUT_DIR	Where the PDF/.SLDDRW pair are written before zipping. Falls back to the Desktop, then a temp folder, if unset/invalid.
DATABASE_URL	PostgreSQL connection string for all master lookup tables.

9. Glossary

Term	Meaning
GAD	General Arrangement Drawing - the overall assembly drawing of the valve.
BOM	Bill of Materials - the uploaded Excel row describing one valve configuration.
COM	Component Object Model - the Windows automation interface SolidWorks exposes.
.SLDDRW	SolidWorks' native drawing document file format.
.SLDBLK	A SolidWorks sketch block file - reusable 2D content inserted onto a sheet.
Resolved dict	The flat, database-free result of <code>resolve_globe_gad()</code> - drawing numbers, dimension values and title-block properties for one BOM row.